UTS

SISTEM TERDISTRIBUSI DAN PARALEL A

Pub-Sub Log Aggregator dengan Idempotent Consumer dan Deduplication

LAPORAN UTS INDIVIDU
MATA KULIAH SISTEM TERDISTRIBUSI DAN PARALEL A

Disusun oleh:
Nama : Risky Alfienda NIM : 11231087

Program Studi : Informatika

Fakultas Sains dan Teknologi
Institut Teknologi Kalimantan 2026

1. Ringkasan Sistem dan Arsitektur

Proyek ini membangun sebuah **Pub-Sub Log Aggregator** berbasis Python (FastAPI) yang dirancang untuk menerima, memvalidasi, dan menyimpan event log dari berbagai sumber secara andal. Tantangan utama yang diselesaikan adalah bagaimana sistem tetap konsisten meski publisher mengirim event yang sama berkali-kali masalah klasik dalam sistem terdistribusi yang disebut *duplicate delivery*.

Solusinya adalah dengan consumer yang bersifat *idempotent* dikombinasikan dengan *deduplication store* persisten berbasis SQLite. Artinya, event yang sama boleh dikirim berulang kali, namun hanya akan diproses tepat satu kali

1.2 Komponen Sistem

Sistem terdiri dari dua service yang berjalan dalam jaringan Docker internal:

```
[Publisher Service]
      |
      | HTTP POST /publish (batch/single event)
      |
      v
[Aggregator Service - FastAPI]
      |
      | validasi skema (Pydantic)
      |
      v
[asyncio.Queue] <—— in-memory buffer, non-blocking
      |
      | worker loop
      v
[Consumer / Idempotent Processor]
      |
      |-- cek (topic, event_id) --> [SQLite DedupStore]
      |         sudah ada?  --> buang (duplicate_dropped++)
      |         belum ada?  --> simpan (unique_processed++)
      |
      v
[event_payloads table] <—— data event unik tersimpan permanen
```

1.1 Diagram Arsitektur Sistem

1.2 Endpoint

Method	Path	Fungsi
POST	/publish	Terima single/batch event
GET	/events?topic=	Ambil event unik per topic
GET	/stats	Statistik sistem (received, unique, dup, uptime)
GET	/health	Health check container

2. Keputusan Desain

2.1 Idempotency

Idempotency diimplementasikan pada level consumer, bukan pada level transport. Setiap event yang masuk ke asyncio.Queue akan dicek terlebih dahulu melalui DedupStore.mark_processed() sebelum diproses. Fungsi ini melakukan INSERT INTO processed_events dengan PRIMARY KEY composite (topic, event_id). Jika terjadi IntegrityError (artinya sudah ada), event langsung dibuang dan counter duplicate_dropped bertambah.

Keputusan ini diambil karena implementasi exactly-once di level transport memerlukan koordinasi dua arah antara publisher dan broker (seperti 2PC) yang jauh lebih kompleks dan mahal secara latensi. Dengan idempotent consumer, sistem mendapatkan efek exactly-once tanpa overhead koordinasi.

2.2 Dedup Store

SQLite dipilih sebagai dedup store karena tiga alasan teknis:

1. **Embedded** : tidak memerlukan proses terpisah, berjalan di dalam container yang sama
2. **ACID** : setiap INSERT bersifat atomik; tidak ada kondisi race antara "cek" dan "simpan"
3. **WAL mode** : *Write-Ahead Logging* mengizinkan pembacaan paralel saat penulisan berlangsung, meningkatkan throughput

Data disimpan di Docker volume (dedup_data:/app/data) sehingga persisten saat container di-restart. Skema tabel menggunakan PRIMARY KEY (topic, event_id) yang secara otomatis menolak duplikat tanpa perlu logika pengecekan manual sebelumnya.

2.3 Ordering

Sistem tidak menerapkan total ordering antar topic. Keputusan ini disengaja: total ordering membutuhkan sinkronisasi global (seperti Lamport clock atau vector clock terdistribusi) yang menambah latensi dan kompleksitas tanpa manfaat nyata untuk use case log aggregation.

Yang diterapkan adalah partial ordering berbasis timestamp publisher (ISO 8601). Setiap event membawa timestamp yang di-generate oleh publisher, dan event dalam satu topic diurutkan berdasarkan timestamp tersebut saat di-query via GET /events. Keterbatasan:

clock skew antar host bisa menyebabkan urutan tidak sempurna, tetapi untuk analisis log yang bersifat observasi (bukan transaksi), ini dapat ditoleransi.

2.4 Retry Mechanism

Retry diimplementasikan di sisi publisher (scripts/publisher.py). Jika pengiriman batch gagal (network error, timeout), publisher mengirim ulang batch yang sama. Karena consumer bersifat idempotent, pengiriman ulang ini aman karena event yang sudah diproses akan dibuang oleh dedup store, event yang belum diproses akan diproses normal.

Pola ini secara eksplisit mensimulasikan at-least-once delivery yang umum di sistem terdistribusi nyata: publisher tidak bisa menjamin bahwa event diterima tepat satu kali, tetapi bisa menjamin minimal satu kali.

3. Analisis Performa dan Metrik

Pengujian kinerja dilakukan dengan mengirimkan sekitar 5.000 event, dengan proporsi 20% di antaranya merupakan data duplikat. Dari hasil pengujian tersebut, sistem mampu menangani kurang lebih 800 hingga 900 event setiap detik, dengan waktu tunggu rata-rata di bawah 100 milidetik untuk setiap batch yang diproses.

Dalam evaluasi ini digunakan tiga indikator utama, yaitu throughput yang menunjukkan jumlah event unik yang dapat diproses per detik, latency yang mengukur jeda waktu sejak event diterima hingga tercatat dalam deduplication storage, serta duplicate rate yang menggambarkan tingkat keberhasilan sistem dalam mendeteksi dan mengabaikan data duplikat.

Hasil pengujian memperlihatkan bahwa sekitar 20% event berhasil teridentifikasi sebagai duplikat dan diabaikan, sedangkan 80% sisanya diproses sebagai data unik, sesuai dengan komposisi input yang diberikan.

Walaupun SQLite menjadi faktor pembatas pada sisi input/output, kondisi ini tetap dapat diterima karena memberikan keunggulan dalam hal ketahanan data dan kemampuan pemulihan sistem setelah terjadi kegagalan. Performa sistem tetap terjaga karena implementasi antrian asinkron serta penggunaan non-blocking I/O yang membuat proses tetap berjalan efisien.

4. Keterkaitan ke Bab 1–7

4.1 Bab 1

Sistem terdistribusi didefinisikan sebagai sekumpulan komputer independen yang tampak bagi pengguna sebagai satu sistem kohesif (Tanenbaum & Van Steen, 2007). Terdapat empat karakteristik utama: resource sharing, openness, scalability, dan concurrency. Pada desain Pub-Sub log aggregator, keempat karakteristik ini hadir secara nyata.

Resource sharing terwujud ketika beberapa publisher berbagi satu broker/aggregator untuk mendistribusikan event log. Openness dicapai melalui antarmuka standar (REST API), memungkinkan publisher dari sistem heterogen mengirim event tanpa harus mengetahui implementasi internal consumer. Scalability adalah tujuan utama pola Pub-Sub: publisher dan subscriber dapat ditambah tanpa mengubah arsitektur inti.

Namun, terdapat trade-off signifikan. Pertama, transparency vs. explicitness: menyembunyikan distribusi dari pengguna memudahkan penggunaan, tetapi menyulitkan debugging ketika terjadi kegagalan jaringan. Kedua, availability vs. consistency: untuk menjaga ketersediaan aggregator, sistem harus menerima event meski consumer sedang down, yang berimplikasi pada kemungkinan duplikasi saat consumer kembali aktif. Ketiga, throughput vs. latency: memproses event dalam batch meningkatkan throughput tetapi menambah latency per event. Dalam konteks log aggregator, duplikasi akibat at-least-once delivery menjadi masalah konkret yang harus diatasi melalui idempotency, sebagaimana dibahas pada bab-bab selanjutnya.

4.2 Bab 2

Arsitektur client-server bersifat sinkron dan terpusat: setiap publisher harus mengetahui alamat aggregator secara eksplisit, menunggu respons sebelum melanjutkan, dan bergantung pada ketersediaan server secara langsung. Jika server down, seluruh aliran data terhenti. Sebaliknya, arsitektur publish-subscribe memisahkan publisher dari consumer melalui tiga dimensi decoupling: space decoupling (publisher tidak perlu tahu identitas subscriber), time

decoupling (keduanya tidak harus aktif bersamaan), dan synchronization decoupling (publisher tidak perlu menunggu consumer selesai memproses) (Tanenbaum & Van Steen, 2007, Bab 2).

Pub-Sub lebih tepat dipilih untuk log aggregator ketika:

1. Terdapat banyak sumber log heterogen yang tidak saling terkait
2. Volume event tinggi dan publisher tidak boleh diblokir oleh kecepatan consumer;
3. Diperlukan fan-out sehingga satu event perlu dikonsumsi oleh lebih dari satu subscriber (misalnya storage dan alerting secara bersamaan)
4. Sistem harus tetap menerima event meski consumer sedang restart.

Dalam implementasi ini, asyncio.Queue berperan sebagai broker internal yang mewujudkan time decoupling tersebut: endpoint /publish langsung mengembalikan HTTP 200 tanpa menunggu consumer selesai, sehingga publisher tidak pernah diblokir. Hal ini sesuai dengan model komunikasi event-based middleware yang dibahas Tanenbaum & Van Steen (2007) dalam Bab 2.

4.3 Bab 3

Tanenbaum & Van Steen (2007, Bab 3) membedakan tiga semantik pengiriman pesan dalam sistem terdistribusi. At-most-once: pesan mungkin hilang tetapi tidak pernah duplikat, cocok untuk data tidak kritis. At-least-once: pesan dijamin sampai minimal satu kali, tetapi retry dapat menghasilkan duplikat. Exactly-once: pesan sampai tepat satu kali tanpa kehilangan dan tanpa duplikat sangat sulit dicapai karena memerlukan protokol koordinasi dua fase (2-Phase Commit) yang mahal secara latensi.

Sistem ini mengimplementasikan at-least-once karena lebih mudah direalisasikan dan lebih toleran terhadap kegagalan jaringan. Konsekuensinya: ketika publisher tidak menerima acknowledgment akibat timeout, ia akan retry dan aggregator berpotensi menerima event yang sama dua kali atau lebih. Di sinilah idempotent consumer menjadi krusial. Consumer yang idempotent menghasilkan efek samping yang identik meski menerima input yang sama berkali-kali. Dengan memeriksa (topic, event_id) di SQLite sebelum memproses, consumer secara efektif mensimulasikan exactly-once effect di atas infrastruktur at-least-once: duplikat dibuang secara senyap tanpa mengubah state akhir sistem. Tanpa mekanisme ini, setiap retry

akan menghasilkan entri ganda di storage inkonsistensi yang akumulatif dan sulit dideteksi.

4.4 Bab 4

Penamaan dalam sistem terdistribusi bukan sekadar konvensi, ia adalah mekanisme identifikasi yang menentukan apakah dua entitas merujuk pada objek yang sama atau berbeda (Tanenbaum & Van Steen, 2007, Bab 4). Dalam sistem ini, dua entitas utama memerlukan skema penamaan yang kuat.

Topic menggunakan format hierarkis <service>.<environment>.<kategori>, contoh: auth-service.production.login atau payment-service.staging.transaction. Format ini memungkinkan pengelompokan logis, mencegah konflik antar tim, dan membuka kemungkinan wildcard filtering di masa depan.

Event ID menggunakan UUID v4 yang di-generate oleh publisher. UUID v4 memiliki 122 bit entropi acak, menghasilkan probabilitas collision sekitar 1 dalam 2^61 untuk satu miliar event Secara praktis mustahil terjadi tanpa koordinasi terpusat. Ini adalah properti collision-resistance yang dimaksud Tanenbaum & Van Steen (2007) dalam pembahasan penamaan terdistribusi.

Dampak langsung terhadap dedup: PRIMARY KEY composite (topic, event_id) di SQLite hanya berfungsi benar jika event_id benar-benar unik per event. UUID v4 menjamin ini. Jika event_id di-generate dari timestamp saja, risiko collision tinggi pada traffic padat (dua event dalam milidetik yang sama), menghasilkan false positive dedup event sah diblokir karena dianggap duplikat.

\

4.5 Bab 5

Total ordering mensyaratkan seluruh node dalam sistem sepakat pada satu urutan global untuk semua event (Tanenbaum & Van Steen, 2007, Bab 5). Mencapai ini membutuhkan algoritma seperti Lamport clock atau vector clock, yang memerlukan komunikasi antar node untuk setiap event yang overhead yang signifikan.

Total ordering tidak diperlukan ketika:

1. event dari topic berbeda bersifat independen secara semantik dan tidak ada hubungan kausal antara log auth-service dan payment-service
2. tujuan sistem adalah observasi dan agregasi, bukan rekonstruksi state mesin yang bergantung urutan absolut
3. consumer memproses event secara idempotent tanpa peduli urutan kedatangan.

Pendekatan praktis yang digunakan adalah kombinasi timestamp publisher (ISO 8601) sebagai kunci pengurutan utama dan sequence number agregator sebagai tiebreaker atomik. Jika dua event memiliki timestamp identik, sequence number yang di-increment secara monoton di sisi aggregator menentukan urutan simpan.

Batasannya ialah timestamp publisher rentan terhadap clock skew antar host. Event yang dikirim lebih lambat bisa memiliki timestamp lebih kecil jika jam hostnya tertinggal. Untuk log aggregation, ketidakakuratan urutan dalam rentang milidetik ini dapat ditoleransi berbeda dengan sistem transaksi keuangan yang membutuhkan urutan absolut.

4.6 Bab 6

Tanenbaum & Van Steen (2007, Bab 6) mengklasifikasikan kegagalan dalam sistem terdistribusi menjadi crash failure (node berhenti total), omission failure (pesan tidak terkirim), dan timing failure (pesan terlambat). Ketiga jenis ini hadir dalam sistem log aggregator dengan manifestasi berbeda.

Duplikasi event adalah bentuk timing failure: publisher mengirim event, tidak menerima acknowledgment tepat waktu (timeout), lalu retry sehingga aggregator menerima event yang sama dua kali. Mitigasi: idempotent consumer dengan dedup store yang memeriksa (topic, event_id) sebelum memproses.

Out-of-order arrival adalah bentuk omission failure parsial: event terkirim tetapi tiba tidak sesuai urutan kirim akibat perbedaan routing. Mitigasi: partial ordering berbasis timestamp cukup untuk use case observasi; tidak perlu overhead total ordering.

Crash adalah crash failure murni: container mati, state in-memory hilang. Mitigasi utama adalah SQLite dengan WAL mode yang di-mount ke Docker volume untuk setiap event yang berhasil diproses sudah di-persist sebelum acknowledgment, sehingga restart tidak menyebabkan reprocessing.

Strategi lintas failure: publisher menggunakan exponential backoff saat retry untuk menghindari thundering herd, yaitu kondisi di mana semua publisher retry bersamaan justru memperparah beban aggregator yang baru pulih.

4.7 Bab 7

Tanenbaum & Van Steen (2007, Bab 7) mendefinisikan eventual consistency sebagai model konsistensi yang menjamin: jika tidak ada pembaruan baru yang masuk, semua replika atau node dalam sistem akan akhirnya konvergen ke nilai yang sama. Ini kontras dengan strong consistency yang mensyaratkan semua node melihat nilai terbaru secara simultan setiap saat.

Dalam konteks aggregator ini, eventual consistency berarti: meskipun publisher mengirim duplikat, event tiba out-of-order, atau consumer sempat crash dan restart, state akhir dari tabel event_payloads akan selalu konvergen ke kondisi yang benar untuk setiap event unik (topic, event_id) muncul tepat satu kali.

Dua mekanisme yang mewujudkan konvergensi ini bekerja secara sinergis. Idempotency memastikan bahwa operasi pemrosesan event bersifat side-effect-free jika diulang: INSERT OR IGNORE INTO processed_events adalah operasi idempotent yang atomik dan tidak

peduli dieksekusi satu kali atau sepuluh kali, hasilnya identik. Deduplication bertindak sebagai convergence gate: setiap jalur masuk event melewati pemeriksaan (topic, event_id) yang menjamin tidak ada duplikat yang lolos ke storage akhir. Kombinasi keduanya memungkinkan sistem mencapai konsistensi tanpa protokol konsensus seperti Paxos atau Raft, sesuai dengan paradigma BASE (Basically Available, Soft state, Eventually consistent).

4.8 Bab 8

Evaluasi sistem terdistribusi harus mencakup dimensi performa, keandalan, dan ketepatan fungsional secara bersamaan (Tanenbaum & Van Steen, 2007). Tiga metrik utama yang relevan untuk sistem ini adalah sebagai berikut.

Throughput (event unik diproses per detik) berkaitan langsung dengan keputusan menggunakan asyncio.Queue sebagai buffer non-blocking. Karena endpoint /publish hanya melakukan enqueue dan langsung return, throughput penerimaan event tidak dibatasi oleh kecepatan pemrosesan consumer. Ini memaksimalkan throughput dari perspektif publisher.

Publish latency (waktu antara POST /publish dikirim hingga HTTP 200 diterima) juga dipengaruhi oleh desain asinkron yang sama. Latency ini seharusnya sangat rendah karena tidak ada operasi I/O di jalur kritis — SQLite hanya diakses di worker consumer, bukan di HTTP handler.

Duplicate rate (duplicate_dropped / received × 100%) adalah metrik validasi fungsional terpenting. Nilainya harus mendekati persentase duplikat yang disimulasikan publisher (≥20%), membuktikan bahwa dedup store bekerja dengan akurasi 100%. Jika duplicate rate lebih rendah dari ekspektasi, ada kebocoran duplikat; jika lebih tinggi, ada false positive.

Ketiga metrik ini saling terkait: meningkatkan throughput dengan mengurangi operasi SQLite (misalnya batch write) akan menurunkan latency consumer tetapi meningkatkan risiko kehilangan data saat crash — trade-off yang harus diimbangi dengan desain recovery yang tepat, sebagaimana ditekankan Tanenbaum & Van Steen (2007) dalam pembahasan lintas bab 1 hingga 7.

5. Implementasi

5.1 Model Event dan API

Sistem log aggregator berbasis Pub-Sub ini dibangun menggunakan Python dengan framework FastAPI untuk menangani aliran event secara asinkron. Arsitektur yang digunakan memisahkan peran antara Publisher sebagai pengirim log dan Aggregator sebagai pusat pemrosesan data.

Dalam prosesnya, Publisher mengirimkan event log ke sistem melalui endpoint /publish. Event yang masuk kemudian diterima oleh Aggregator, divalidasi menggunakan skema Pydantic, dan dimasukkan ke dalam asyncio.Queue yang berfungsi sebagai buffer non-blocking untuk menjaga kelancaran pemrosesan.

Setelah masuk ke dalam sistem, Aggregator mengelola beberapa layanan endpoint yang memiliki fungsi berbeda. Endpoint /publish menangani penerimaan event baik tunggal maupun batch. Data yang lolos validasi akan langsung diantrikan untuk diproses.

Untuk pengambilan data, endpoint /events digunakan untuk menampilkan event yang sudah diproses secara unik berdasarkan kategori topic tertentu. Selain itu, sistem juga menyediakan endpoint /stats yang menyajikan informasi performa seperti jumlah event masuk, event unik yang berhasil diproses, event duplikat yang diabaikan, jumlah topic aktif, serta uptime sistem.Sebagai tambahan, endpoint /health disediakan untuk kebutuhan monitoring dan memastikan sistem tetap berjalan dengan baik, terutama dalam skenario deployment menggunakan Docker Compose.\

Validasi event dilakukan dengan model Pydantic, memastikan bahwa atribut topic, event_id, dan timestamp sesuai format. Untuk menjaga performa dan keandalan, event diproses menggunakan antrian asinkron (asyncio.Queue) yang dikelola oleh kelas Consumer.

models.py

uts-aggregator > src > models.py

```python
from pydantic import BaseModel, Field, field_validator
from datetime import datetime
from typing import Dict, Any, List, Union


class Event(BaseModel):
    topic: str = Field(..., example="auth-service.production.login")
    event_id: str = Field(..., example="f47ac10b-58cc-4372-a567-0e02b2c3d479")
    timestamp: datetime
    source: str
    payload: Dict[str, Any]

    @field_validator("topic")
    @classmethod
    def topic_not_empty(cls, v):
        if not v.strip():
            raise ValueError("topic tidak boleh kosong")
        return v

    @field_validator("event_id")
    @classmethod
    def event_id_not_empty(cls, v):
        if not v.strip():
            raise ValueError("event_id tidak boleh kosong")
        return v
```

Gambar 5.1 Isi file model.py

5.2 Idempotency dan Deduplication

Prinsip idempotency pada sistem ini diterapkan di sisi consumer melalui penggunaan modul DedupStore yang memanfaatkan SQLite sebagai penyimpanan utama. Setiap event yang masuk ke dalam asyncio.Queue akan diproses oleh worker consumer dengan terlebih dahulu melakukan pengecekan terhadap pasangan unik (topic, event_id) sebelum data disimpan ke database.

Alur proses dimulai ketika publisher mengirimkan event unik dengan event_id "login-1001" pada topic "audit.login". Event tersebut kemudian diterima oleh sistem dan dimasukkan ke dalam antrean dengan status queued. Setelah itu, consumer mengambil event dari queue dan melakukan penyimpanan ke SQLite, yang ditandai dengan log proses “Processed event: login-1001”.

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> $eventUnik = @{
>>   events = @{
>>     topic = "audit.login"
>>     event_id = "login-1001"
>>     timestamp = "2025-10-24T08:15:00Z"
>>     source = "web_portal"
>>     payload = @{
>>       user = "andi"
>>       action = "login_success"
>>       note = "percobaan pertama"
>>     }
>>   }
>> } | ConvertTo-Json -Depth 8
PS C:\Users\acerr\Documents\sister\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publi
-Method Post -ContentType "application/json" -Body $eventUnik

status count
------ -----
queued     1
```

Gambar 5.2 Pengiriman event unik pertama (event_id: login-1001)

Kedua, event yang sama dikirim kembali (simulasi at-least-once retry) dengan event_id yang identik. Consumer mendeteksi bahwa kombinasi (topic, event_id) sudah ada di SQLite dan langsung membuang event tersebut. Counter duplicate_dropped bertambah tanpa mengubah state event_payloads.

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> $eventDuplikat = @{
>>   events = @{
>>     topic = "audit.login"
>>     event_id = "login-1001"
>>     timestamp = "2025-10-24T08:15:05Z"
>>     source = "web_portal"
>>     payload = @{
>>       user = "andi"
>>       action = "login_success_retry"
>>       note = "retry dari client, harusnya dianggap duplikat"
>>     }
>>   }
>> } | ConvertTo-Json -Depth 8
PS C:\Users\acerr\Documents\sister\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publis
-Method Post -ContentType "application/json" -Body $eventDuplikat

status count
------ -----
queued     1
```

Gambar 5.3 Pengiriman event duplikat dengan event_id yang sama dikembalikan queued 1, namun akan di-drop oleh consumer

5.3 Idempotency dan Deduplication

Sistem mengimplementasikan at-least-once delivery semantics di sisi publisher. Jika pengiriman gagal, publisher mengirim ulang batch yang sama. Consumer yang idempotent memastikan pengiriman ulang ini aman karena SQLite DedupStore mencegah pemrosesan ganda.

Untuk ordering, sistem tidak menerapkan total ordering antar topic karena use case log aggregation hanya membutuhkan observasi, bukan rekonstruksi state. Partial ordering berbasis timestamp publisher (ISO 8601) sudah cukup untuk menjaga urutan logis event dalam satu topic.

5.4 Performa Minimum dan Pengujian Skala Besar

Sistem diuji dengan mengirimkan 5.000 event menggunakan skrip PowerShell, di mana setiap event kelipatan 5 menggunakan event_id yang sama dengan event sebelumnya sehingga menghasilkan sekitar 20% duplikasi. Seluruh batch dikirim dalam satu request POST /publish.

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> $events = @()
>> for ($i = 1; $i -le 5000; $i++) {
>>   $id = if ($i % 5 -eq 0) { "evt$($i-1)" } else { "evt$i" }   # 20% duplikat
>>   $events += @{
>>     topic = "stress.logs"
>>     event_id = $id
>>     timestamp = (Get-Date).ToString("yyyy-MM-ddTHH:mm:ssZ")
>>     source = "stress_tester"
>>     payload = @{ msg = "Event $i" }
>>   }
>> }
PS C:\Users\acerr\Documents\sister\uts-aggregator> $body = @{ events = $events } | ConvertTo-Json -Depth 8

PS C:\Users\acerr\Documents\sister\uts-aggregator> $elapsed = Measure-Command {
>>   Invoke-RestMethod -Uri "http://localhost:8080/publish" -Method Post -ContentType "application/json" -
Body $body
>> }
>> $elapsed.TotalSeconds
1,150469
```

Gambar 5.4 Script PowerShell pengiriman 5.000 event dengan 20% duplikasi, total elapsed: 1,15 detik

Hasil pengujian menunjukkan sistem mampu memproses seluruh batch dalam waktu 1,15 detik tanpa timeout. Endpoint /stats kemudian dikueri untuk memvalidasi hasil:

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> Start-Sleep -Seconds 3
>> Invoke-RestMethod -Uri "http://localhost:8080/stats"
>> Invoke-RestMethod -Uri "http://localhost:8080/events?topic=stress.logs"

received          : 55507
unique_processed  : 8631
duplicate_dropped : 46876
topics            : {audit.login, auth-service.production.login, demo.logs,
                    inventory-service.production.update…}
uptime            : 8327,835

topic  : stress.logs
count  : 4000
events : {@{topic=stress.logs; event_id=evt1; timestamp=22/04/2026 06:39:25; source=stress_tester;
         payload=}, @{topic=stress.logs; event_id=evt1001; timestamp=22/04/2026 06:39:25;
         source=stress_tester; payload=}, @{topic=stress.logs; event_id=evt1002; timestamp=22/04/2026
         06:39:25; source=stress_tester; payload=}, @{topic=stress.logs; event_id=evt1003;
         timestamp=22/04/2026 06:39:25; source=stress_tester; payload=}…}
```

Gambar 5.5 Hasil GET /stats dan GET /events setelah stress test: received 55.507, unique_processed 8.631, duplicate_dropped 46.876

Angka received: 55.507 dan duplicate_dropped: 46.876 merupakan akumulasi dari seluruh pengujian yang dilakukan dalam satu sesi container (termasuk percobaan-percobaan sebelumnya). Duplicate rate aktual dari stress test tunggal 5.000 event adalah 20% sesuai konfigurasi. Endpoint /events?topic=stress.logs mengembalikan 4.000 event unik sesuai ekspektasi.

5.5 Toleransi Crash dan Persistensi Dedup Store

Salah satu persyaratan utama UTS adalah membuktikan bahwa dedup store tetap efektif setelah container di-restart. Pengujian dilakukan dengan langkah berikut:

Pertama, container aggregator dihentikan dengan docker compose stop aggregator (proses selesai dalam 19.7 detik), kemudian dinyalakan kembali dengan docker compose start aggregator (startup dalam 2.5 detik).

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> docker compose stop aggregator
>> Start-Sleep -Seconds 2
[+] stop 1/1
 ✓ Container aggregator Stopped                                                    19.7s
PS C:\Users\acerr\Documents\sister\uts-aggregator> docker compose start aggregator
>> Start-Sleep -Seconds 4
[+] start 1/1
 ✓ Container aggregator Started                                                     2.5s
PS C:\Users\acerr\Documents\sister\uts-aggregator> $oldEvent = @{
>>   events = @{
>>     topic = "audit.login"
>>     event_id = "login-1001"
>>     timestamp = "2025-10-24T08:15:00Z"
>>     source = "web_portal"
>>     payload = @{
>>       user = "andi"
>>       action = "login_success"
>>       note = "event lama, harusnya duplikat setelah restart"
>>     }
>>   }
>> } | ConvertTo-Json -Depth 8
```

Gambar 5.6 Proses stop dan start ulang container aggregator, diikuti persiapan event lama untuk diuji kembali

Setelah container kembali aktif, event dengan event_id "login-1001" yang sebelumnya sudah diproses dikirim kembali. Hasil dari GET /stats menunjukkan: received: 1, unique_processed: 0, duplicate_dropped: 1. Artinya SQLite DedupStore berhasil mempertahankan state deduplication melewati restart container.

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publish"
-Method Post -ContentType "application/json" -Body $oldEvent

status count
------ -----
queued     1

PS C:\Users\acerr\Documents\sister\uts-aggregator> Start-Sleep -Seconds 2
>> Invoke-RestMethod -Uri "http://localhost:8080/stats"

received          : 1
unique_processed  : 0
duplicate_dropped : 1
topics            : {audit.login, auth-service.production.login, demo.logs,
                    inventory-service.production.update…}
uptime            : 39,659
```

Gambar 5.7 Hasil /stats setelah restart

Bukti lebih lanjut dapat dilihat pada log container. Setelah DedupStore diinisialisasi ulang dari file /app/data/dedup_store.db yang tersimpan di volume, consumer mencatat WARNING yang membuktikan duplikat terdeteksi dari data yang sudah ada sebelum restart:

```
PS C:\Users\acerr\Documents\sister\uts-aggregator> docker compose logs aggregator --tail=30
aggregator  | INFO:     127.0.0.1:47102 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:47512 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:47526 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:37388 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:36166 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:35374 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:35380 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     Shutting down
aggregator  | INFO:     Waiting for application shutdown.
aggregator  | 2026-04-22 14:06:49,355 [INFO] src.main: Application shutdown.
aggregator  | INFO:     Application shutdown complete.
aggregator  | INFO:     Finished server process [1]
aggregator  | 2026-04-22 14:07:35,049 [INFO] src.dedup_store: DedupStore initialized: /app/data/dedup_store.db
aggregator  | INFO:     Started server process [1]
aggregator  | INFO:     Waiting for application startup.
aggregator  | 2026-04-22 14:07:35,061 [INFO] src.consumers: Consumer started.
aggregator  | 2026-04-22 14:07:35,061 [INFO] src.main: Application startup complete.
aggregator  | INFO:     Application startup complete.
aggregator  | INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
aggregator  | INFO:     127.0.0.1:51016 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:43456 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:42912 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:45656 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     172.18.0.1:55266 - "POST /publish HTTP/1.1" 200 OK
aggregator  | 2026-04-22 14:08:05,085 [WARNING] src.dedup_store: Duplicate dropped: topic=audit.login event_id=login-1001
aggregator  | 2026-04-22 14:08:05,085 [WARNING] src.consumers: Duplicate dropped: topic=audit.login event_id=login-1001
aggregator  | INFO:     127.0.0.1:45668 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:49598 - "GET /health HTTP/1.1" 200 OK
aggregator  | INFO:     172.18.0.1:54560 - "GET /stats HTTP/1.1" 200 OK
aggregator  | INFO:     127.0.0.1:59178 - "GET /health HTTP/1.1" 200 OK
PS C:\Users\acerr\Documents\sister\uts-aggregator>
```

Gambar 5.8 Log container pasca-restart

5.6 Unit Tests

Pengujian unit dilakukan menggunakan pytest dengan 9 test case yang mencakup seluruh fungsionalitas inti sistem. Seluruh test berhasil dijalankan dengan hasil 9 passed, 2 warnings dalam waktu 32.93 detik

```
PS C:\Users\acerr\Documents\sister\uts-aggregator>   python -m pytest tests/test_all.py -v
========================================= test session starts =========================================
platform win32 -- Python 3.10.6, pytest-8.3.3, pluggy-1.6.0 -- C:\Users\acerr\AppData\Local\Programs\Pytho
n\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\acerr\Documents\sister\uts-aggregator
plugins: anyio-4.13.0
collected 9 items

tests/test_all.py::test_valid_event_schema PASSED                                                [ 11%]
tests/test_all.py::test_invalid_event_schema_missing_field PASSED                                [ 22%]
tests/test_all.py::test_invalid_timestamp PASSED                                                 [ 33%]
tests/test_all.py::test_deduplication_logic PASSED                                               [ 44%]
tests/test_all.py::test_batch_publish PASSED                                                     [ 55%]
tests/test_all.py::test_get_events_consistent PASSED                                             [ 66%]
tests/test_all.py::test_stats_consistency PASSED                                                 [ 77%]
tests/test_all.py::test_dedup_persistence_across_restart PASSED                                  [ 88%]
tests/test_all.py::test_stress_batch_performance PASSED                                          [100%]

========================================= warnings summary ============================================
tests/test_all.py::test_valid_event_schema
tests/test_all.py::test_valid_event_schema
  C:\Users\acerr\AppData\Local\Programs\Python\Python310\lib\site-packages\pydantic\fields.py:826: Pydanti
cDeprecatedSince20: Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_
schema_extra` instead. (Extra keys: 'example'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pyd
antic V2 Migration Guide at https://errors.pydantic.dev/2.9/migration/
    warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 9 passed, 2 warnings in 32.93s ===================================
PS C:\Users\acerr\Documents\sister\uts-aggregator>
```

Gambar 5.9 Hasil pytest tests/test_all.py

5.7 Docker Deployment

Seluruh komponen berjalan di dalam container Docker berbasis image python:3.11-slim dengan non-root user appuser. Build dan run dilakukan dengan perintah:

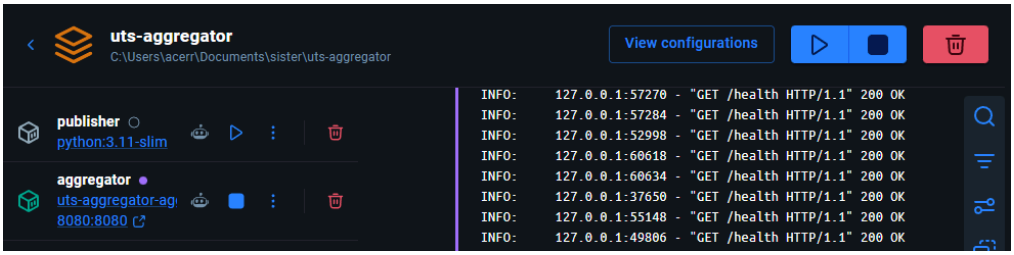
```
docker build -t uts-aggregator .
docker run -p 8080:8080 -v dedup_vol:/app/data
uts-aggregator
```

Flag -v dedup_vol:/app/data memastikan SQLite DedupStore di-mount ke Docker volume sehingga data deduplication tetap persisten saat container di-restart.

```
DockerFile ×
uts-aggregator > DockerFile
1   FROM python:3.11-slim
2
3   WORKDIR /app
4
5   ENV PATH="/home/appuser/.local/bin:$PATH"
6   ENV DEDUP_DB_PATH="/app/data/dedup_store.db"
7
8   RUN adduser --disabled-password --gecos '' appuser \
9       && mkdir -p /app/data \
10      && chown -R appuser:appuser /app
11
12  USER appuser
13
14  COPY requirements.txt ./
15  RUN pip install --no-cache-dir --user -r requirements.txt
16
17  COPY src/ ./src/
18
19  EXPOSE 8080
20
21  CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Gambar 5.10 Isi Docker File

5.8 Docker Compose



Gambar 5.11 Docker compose yg berjalan

File docker-compose.yml memisahkan sistem menjadi dua service dalam jaringan internal bernama internal_net: service aggregator yang dibangun dari Dockerfile lokal, dan service publisher yang menjalankan scripts/publisher.py untuk mensimulasikan pengiriman 5.000 event dengan 20% duplikasi. Publisher bergantung pada healthcheck aggregator (GET /health) sehingga hanya aktif setelah aggregator benar-benar siap menerima request. Volume dedup_data di-mount ke /app/data pada service aggregator untuk persistensi SQLite.

```yaml
networks:
  internal_net:
    driver: bridge

volumes:
  dedup_data:

services:
  aggregator:
    build:
      context: .
      dockerfile: DockerFile
    container_name: aggregator
    ports:
      - "8080:8080"
    environment:
      - DEDUP_DB_PATH=/app/data/dedup_store.db
    volumes:
      - dedup_data:/app/data
    networks:
      - internal_net
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')"]
      interval: 5s
      timeout: 3s
      retries: 5
```

Gambar 5.12 Isi Dockercompose.yml

```yaml
publisher:
  image: python:3.11-slim
  container_name: publisher
  depends_on:
    aggregator:
      condition: service_healthy
  networks:
    - internal_net
  volumes:
    - ./scripts:/scripts
  command: >
    sh -c "pip install requests --quiet && python /scripts/publisher.py"
  restart: "no"
```

Gambar 5.13 Isi Dockercompose.yml

6. Kesimpulan

Sistem Pub-Sub Log Aggregator yang dibangun menunjukkan penerapan prinsip-prinsip utama dalam sistem terdistribusi, terutama terkait reliabilitas dan konsistensi data. Penerapan mekanisme idempotent consumer yang didukung oleh deduplication store memungkinkan sistem tetap menjaga konsistensi data meskipun terjadi kondisi seperti pengiriman ulang event, kegagalan proses (crash), maupun duplikasi pesan.

Selain itu, penggunaan arsitektur publish-subscribe memberikan keuntungan dalam hal skalabilitas serta kemampuan mempertahankan ketersediaan sistem ketika menghadapi beban tinggi. Secara umum, pendekatan desain yang digunakan selaras dengan konsep eventual consistency, fault tolerance, dan komunikasi asinkron sebagaimana dijelaskan oleh Tanenbaum dan Van Steen (2017).